Distributed System for Near Real-Time Network Anomaly Detection with Spark MLlib on AWS EMR

Contents

1 Problem Statement **3**

1.1 Type of anomaly . . . 3

1.2 Network context . . . 3

1.3 Problem delimitation . . . 3

1.4 Justification . . . 3

1.5 Choice of datasets . . . 4

1.5.1 CIC-IDS2017 — Training and evaluation . . . 4

1.5.2 CSE-CIC-IDS2018 — Streaming demonstration . . . 4

2 Objectives **5**

2.1 General objective . . . 5

2.2 Specific objectives . . . 5

3 Chosen Machine Learning Models and Justification **5**

3.1 Supervised model . . . 5

3.2 Unsupervised model . . . 5

3.3 Justification of the comparison . . . 6

4 Data Ingestion, Cleaning, and Transformation **6**

4.1 Ingestion pipeline . . . 6

4.2 Cleaning and transformation . . . 6

4.3 Feature engineering . . . 6

5 Work Methodology **7**

5.1 Data preparation . . . 7

5.2 Distributed training . . . 7

5.3 Performance evaluation . . . 7

5.3.1 Machine Learning metrics . . . 7

5.3.2 Big Data and distributed systems metrics . . . 8

5.4 Test phase . . . 8

5.5 Demonstration and final validation . . . 8

6 EMR Cluster Topology **8**

7 Apache Spark Configuration Parameters **9**

7.1 Per-node parameter derivation (m5.xlarge: 4 vCPU / 16 GiB RAM) . . . 9

7.2 Full configuration - spark-defaults.conf . . . 9

7.3 Parameter tuning for horizontal scalability experiments . . . 10

8 Project S3 Bucket Structure **10**

9 Spark Structured Streaming Configuration **11**

9.1 End-to-end latency measurement . . . 11

10 Cluster Provisioning **11**

10.1 AWS CLI 11

10.2 emr-spark-config.json file 12

11 AWS Region and Network Configuration **12**

1 Problem Statement

The exponential growth of network traffic in distributed infrastructures and cloud environments has significantly increased the need for monitoring mechanisms and early detection of anomalies and cyberattacks. In high-volume data scenarios, traditional centralized analysis systems present limitations related to scalability, latency, and processing capacity, especially when large amounts of network flows need to be analyzed continuously.

In this context, the present proposal puts forward the development of a distributed system for *near real-time* network traffic anomaly detection using Apache Spark, Spark MLlib, and AWS EMR. The system will be specifically oriented toward detecting anomalies based on network flow behavior (*flow-based anomaly detection*) through machine learning techniques executed on distributed infrastructures.

Near real-time is understood as the continuous processing of data in micro-batches with latencies on the order of seconds, a characteristic feature of Apache Spark Structured Streaming [4].

1.1 Type of anomaly

The project will focus on anomalies related to irregular TCP/IP traffic patterns: abnormal variations in traffic volume, irregular increases in simultaneous connections, behaviors associated with distributed denial of service (DDoS), statistically atypical network flows, and anomalous communication patterns between nodes.

The approach will be limited to the analysis of statistical and temporal characteristics of network flow, without performing deep packet inspection (*Deep Packet Inspection*). This delimitation reduces computational complexity and facilitates integration with distributed massive data processing systems.

1.2 Network context

The proposal is situated within distributed network scenarios with continuous traffic generation and a need for parallel processing. The system is oriented toward contexts where there is a high volume of data generation with continuous traffic, near real-time processing is required, data must be processed through distributed architectures, and horizontal scalability constitutes an operational requirement.

Consequently, the problem is addressed from a Big Data perspective, primarily considering challenges related to volume, velocity, and distributed data processing.

1.3 Problem delimitation

The technical scope of the project is delimited to the distributed processing of network traffic flows, anomaly detection using classical Machine Learning models available in Spark MLlib, distributed training, performance evaluation under batch and streaming scenarios, and the analysis of distributed system metrics.

The project does not contemplate the development of new detection algorithms, implementation of Deep Learning techniques, deep packet inspection, automatic response systems against attacks, or implementation of physical network infrastructure. The proposal focuses mainly on analyzing the behavior and performance of a distributed anomaly detection system within a Big Data context.

1.4 Justification

The growing dependence on distributed systems and cloud architectures has increased the need for tools capable of processing large volumes of network traffic efficiently and scalably. In this context, Big Data technologies offer suitable mechanisms for addressing problems related to massive information processing and continuous data analysis.

Apache Spark provides distributed processing and data stream analysis capabilities through Spark Structured Streaming, while AWS EMR facilitates the administration of scalable environments for distributed execution. The combination of these technologies makes it possible to study the feasibility of anomaly detection systems capable of operating under high-volume and high-velocity data generation constraints.

1.5 Choice of datasets

The project uses two datasets from the Canadian Institute for Cybersecurity (CIC), with differentiated roles within the pipeline: CIC-IDS2017 [1] for training and evaluation, and CSE-CIC-IDS2018 [2] exclusively for the streaming demonstration.

1.5.1 CIC-IDS2017 — Training and evaluation

CIC-IDS2017 contains network traffic captured over five days (July 3–7, 2017), organized into CSV files with more than 80 statistical flow features extracted using CICFlowMeter. It includes benign traffic and multiple attack categories: DoS, DDoS, Brute Force, Web attacks, Botnet, Heartbleed, and Infiltration.

For this project, the files corresponding to three specific days are used, selected for their direct alignment with the anomaly types defined in §1.1:

- **Monday (benign traffic):** Provides the baseline of normal behavior. It constitutes the only training set for the unsupervised model (KMeans), since this model learns exclusively from benign traffic. It is also part of the training set for the supervised model (Random Forest).
- **Wednesday (DoS/DDoS):** Contains Slowloris, Hulk, GoldenEye, Slowhttptest, and Heartbleed attacks, directly representative of volumetric TCP flow anomalies. It is used in supervised training and in calibrating the distance threshold of the KMeans model.
- **Friday (DDoS LOIT, PortScan, Botnet):** Includes the DDoS attack coordinated from multiple IPs (LOIT), the most representative of anomalous communication patterns between nodes. It fulfills the same role as Wednesday: supervised training and calibration of the unsupervised threshold.

The Tuesday and Thursday files are reserved as the batch test set, ensuring that the models are evaluated on instances not seen during training or calibration.

1.5.2 CSE-CIC-IDS2018 — Streaming demonstration

CSE-CIC-IDS2018 is the direct successor to CIC-IDS2017, generated on a larger-scale infrastructure: 50 attacking machines and a victim network with five departments, 420 machines, and 30 servers. It shares the general CICFlowMeter feature schema with CIC-IDS2017, although with minor differences in column names that require a prior harmonization step.

This set will be used *exclusively* during the streaming demonstration phase for two fundamental reasons. First, the models trained on CIC-IDS2017 will never have processed CIC-IDS2018 data, which guarantees that the demonstration constitutes a genuine evaluation on data completely unknown to the model. Second, the dataset is hosted in a public AWS S3 bucket, which allows it to be consumed from the EMR cluster without the need for intermediate transfers, resulting in native integration with the project's architecture.

For the demonstration, the files from February 20 and 21, 2018 will be used, which contain the DDoS-LOIC (HTTP and UDP) and DDoS-HOIC attacks executed from ten simultaneous attacking machines, an ideal scenario for validating the detection of volumetric anomalies under continuous flow conditions.

2 Objectives

2.1 General objective

Develop and evaluate a distributed near real-time network traffic anomaly detection system using Apache Spark, Spark MLlib, and AWS EMR, oriented toward scalable data processing within a Big Data context.

2.2 Specific objectives

1. Implement a distributed pipeline for network traffic processing using Apache Spark Structured Streaming and Spark DataFrames.
2. Train supervised and unsupervised anomaly detection models using Spark MLlib on the selected CIC-IDS2017 datasets.
3. Compare the behavior of supervised and unsupervised models within a distributed environment, analyzing differences in predictive performance, training time, and horizontal scalability.
4. Evaluate system performance using Machine Learning metrics for the supervised model and Big Data system metrics for both models.
5. Analyze system behavior under batch and near real-time scenarios using Spark Structured Streaming.
6. Demonstrate the integrated operation of the distributed pipeline processing continuous streams from CSE-CIC-IDS2018 on AWS S3, quantifying throughput and end-to-end detection latency on data not seen during training.

3 Chosen Machine Learning Models and Justification

The proposal contemplates the use of two complementary machine learning approaches: a supervised model based on Random Forest and an unsupervised model based on KMeans with centroid distance scoring. Both are natively available in Spark MLlib [3], which guarantees their distributed execution on the EMR cluster without external dependencies.

The selection of both approaches aims to compare the behavior of supervised and unsupervised models within a distributed massive data processing environment.

3.1 Supervised model

The supervised model will use Random Forest implemented via Spark MLlib. This model was selected for its robustness against noisy data, its ability to work with large volumes of data in a parallelizable way, its documented good performance in network traffic classification tasks, and its relative interpretability compared to more complex models.

The supervised approach allows the system to be trained using known labels for benign and malicious traffic, facilitating quantitative performance evaluation through classic classification metrics.

3.2 Unsupervised model

As a complement, KMeans implemented in Spark MLlib will be used, applied as an anomaly detector through centroid distance scoring. The model will be trained exclusively on benign traffic (Monday of CIC-IDS2017), so that anomalous flows manifest as points far from the learned centroids. During inference, each flow receives a

score proportional to its distance to the nearest centroid; flows that exceed a predefined threshold are classified as anomalous.

This approach is especially relevant in real-world scenarios where labeled data is not always available or where previously unknown attacks may appear, since detection is based on statistical deviations from the learned normal behavior.

3.3 Justification of the comparison

The comparison between supervised and unsupervised models will not focus solely on machine learning metrics, but also on their behavior within the Big Data context. In particular, distributed training time, horizontal scalability, computational costs, inference time, and performance on continuous data streams will be analyzed.

In this way, the project prioritizes the comprehensive analysis of the distributed system over the exclusive optimization of classification metrics.

4 Data Ingestion, Cleaning, and Transformation

4.1 Ingestion pipeline

Data ingestion is performed via `SparkSession` with distributed loading of CSV files directly from the configured paths, without automatic schema inference, allowing explicit control of type conversion in a later stage and avoiding incorrect inferences caused by corrupt values present in the CICFlowMeter files. The training and test sets are loaded independently from files corresponding to specific days of CIC-IDS2017, respecting the dataset's temporal distribution given that each day contains distinct traffic patterns. Traceability columns `_source_file` and `_split` are added to each row, and the union of multiple files within each set is performed using `unionByName`, which aligns columns by name rather than by position, ensuring consistency in the face of possible variations in column order between files.

4.2 Cleaning and transformation

In order to guarantee the quality and consistency of the data throughout the pipeline, null, infinite, and duplicate values present in the dataset were removed, since their presence can introduce bias into model training and degrade the performance of the distributed system. The data was subsequently split into training and test sets from files corresponding to specific days of CIC-IDS2017, a separation that facilitates the management and understanding of the pipeline by respecting the dataset's temporal and thematic distribution, ensuring that the models are evaluated on instances not seen during training.

4.3 Feature engineering

Feature engineering constitutes the final stage of data preparation, aimed at transforming the dataset's variables into a standardized numerical representation compatible with the machine learning algorithms available in Spark MLlib. First, the textual traffic labels were converted into numeric indices in descending order of frequency, so that the most frequent class, corresponding to benign traffic, receives index 0.0, which facilitates the interpretation of results during model evaluation. Next, the resulting numeric columns were assembled into a single feature vector in deterministic alphabetical order, guaranteeing pipeline reproducibility regardless of the underlying file system. Finally, this vector was normalized by centering each dimension to zero mean and unit standard deviation, a step of particular relevance for the unsupervised KMeans model, whose Euclidean distance metric is sensitive to variable scale. In order to preserve the methodological validity of the results

and avoid information leakage between sets, all statistical parameters derived from these transformations were computed exclusively on the training set and applied consistently to the test set, ensuring that no information from instances unseen during training influenced the models' learning process.

5 Work Methodology

The proposed methodology is based on a distributed processing workflow using Apache Spark, Spark MLlib, and AWS EMR as the distributed execution platform. The main objective is to evaluate the behavior of network anomaly detection models within a Big Data environment capable of supporting parallel processing and near real-time data analysis.

The methodology is directly related to the problem statement, given that the high volume and velocity of network traffic generation require scalable distributed processing and analysis mechanisms.

5.1 Data preparation

The preparation of the datasets constitutes the foundation of the distributed pipeline and is described in detail in Section 4. From a methodological standpoint, this stage comprises the distributed loading of CIC-IDS2017 CSV files via Spark DataFrames, followed by the transformations needed to build the training, validation, and test sets. For the streaming demonstration, an additional column-harmonization stage will be performed between CIC-IDS2017 and CSE-CIC-IDS2018, given that both datasets share the general CICFlowMeter schema but present minor differences in variable names. Distributed processing will make it possible to reduce execution times and guarantee the system's scalability.

5.2 Distributed training

Subsequently, the selected models will be trained using Spark MLlib on a distributed environment managed via AWS EMR. The training workflow contemplates the distribution of data across cluster nodes, the construction of machine learning pipelines using MLlib's `Pipeline` API, parallel training of both models, distributed evaluation of results, and storage of the trained models in S3 for reuse during the streaming phase.

The use of Spark MLlib makes it possible to take advantage of parallelization and distributed processing mechanisms built into Apache Spark, facilitating the efficient handling of large datasets.

5.3 Performance evaluation

The project's evaluation will consider two dimensions: the predictive performance of the Machine Learning models and the performance of the distributed system. Since the project is primarily situated within the Big Data context, methodological priority will be oriented toward analyzing the behavior of the distributed system.

5.3.1 Machine Learning metrics

Machine learning metrics make it possible to evaluate each model's anomaly detection capability, but their applicability differs depending on the approach. For the supervised model (Random Forest), the dataset labels are available during training, so metrics such as Accuracy, Precision, Recall, F1-score, and False Positive Rate are calculated directly on the predictions. For the unsupervised model (KMeans), the CIC-IDS2017 labels *are not used during training*; however, they are used *post-hoc* in the evaluation phase to quantify the extent to which the centroid distance scores separate benign from malicious traffic, allowing an objective comparison between both approaches.

These metrics will be used primarily as a functional validation mechanism for the system.

5.3.2 Big Data and distributed systems metrics

The main evaluation axis will be oriented toward analyzing distributed performance. Processing throughput, end-to-end detection latency, distributed training time, per-micro-batch inference time, horizontal scalability under variations in the number of cluster nodes, computational resource usage (CPU, memory, S3 I/O), and the system's sustained behavior under continuous processing via Spark Structured Streaming will all be measured.

The prioritization of these metrics is justified because the project's central objective is to analyze the feasibility and efficiency of a distributed anomaly detection system within massive data processing scenarios.

5.4 Test phase

The test phase will validate the behavior of the trained models on the Tuesday and Thursday files of CIC-IDS2017, reserved exclusively for this purpose. Each model's anomaly detection capability, the stability of the distributed system under different processing loads, and behavior in the face of attack categories not present in training (Web attacks and Infiltration from Thursday) will be analyzed. The experimental evaluation will make it possible to establish relationships between predictive performance and computational efficiency within the distributed context.

5.5 Demonstration and final validation

The final phase will consist of a comprehensive demonstration of the system operating on continuous data streams from CSE-CIC-IDS2018, consumed directly from AWS S3 via Spark Structured Streaming. The files from February 20 and 21, 2018 (DDoS-LOIC-HTTP, DDoS-LOIC-UDP, and DDoS-HOIC from ten simultaneous attackers) will be streamed into the pipeline, which will apply the models previously trained on CIC-IDS2017 to classify each flow in near real-time.

Since CSE-CIC-IDS2018 was not used at any training or testing stage, this demonstration constitutes a validation on data completely unknown to the model, strengthening the methodological soundness of the results. The native integration between the CIC-IDS2018 S3 bucket and the EMR cluster eliminates the need for intermediate data transfers.

The demonstration will include visualization mechanisms to facilitate the interpretation of system behavior and detection results, with throughput and latency metrics updating continuously.

6 EMR Cluster Topology

Role	EC2 Type	vCPU	RAM(GiB)	Storage (GiB)	Count
Master	m5.xlarge	4	16	64 EBS	1
Core	m5.xlarge	4	16	64 EBS	2 (minimum)

The justification for the m5.xlarge instance as the cluster's base unit follows these criteria:

- Adequate vCPU/RAM ratio for mixed Spark workloads (ML training + Structured Streaming).
- On-demand hourly cost of $0.192 USD, which allows multi-hour experiments within a reasonable academic budget.
- Guaranteed availability in the us-east-1 region, where the public CSE-CIC-IDS2018 bucket is hosted.

- Homogeneous scalability: all nodes share the same instance type, eliminating asymmetries in Spark task distribution.

7 Apache Spark Configuration Parameters

The parameters set out below are the determinants of the distributed engine's behavior. They are derived from the m5.xlarge node's hardware specifications following standard Spark sizing heuristics.

7.1 Per-node parameter derivation (m5.xlarge: 4 vCPU / 16 GiB RAM)

Sizing decision	Value	Justification
Cores per executor	2	Balance between intra-task parallelism and I/O contention
Executors per core node	1	4 vCPU - 1 (YARN NM) = 3 available; 1 executor × 2 cores + 1 free for OS
RAM per executor	6 GiB	16 GiB - 6 GiB (YARN NM + OS) = 12 GiB; 6 GiB per executor + overhead
spark.memory.fraction	0.75	Larger fraction for RF training with large feature vectors
spark.memory.storageFraction	0.40	Cache for training DataFrames reused across multiple epochs

7.2 Full configuration - spark-defaults.conf

```
# Resources per executor
spark.executor.cores                  2
spark.executor.memory                 6g
spark.executor.memoryOverhead         1024

# Driver (Master node m5.xlarge)
spark.driver.cores                    2
spark.driver.memory                   6g
spark.driver.memoryOverhead           512

# Instances (base cluster: 1 master + 2 core)
spark.executor.instances              2

# Parallelism
# Rule: 2-3x total available cores
# Base (2 core nodes x 1 executor x 2 cores = 4 cores) -> 16 partitions
spark.default.parallelism             16
spark.sql.shuffle.partitions          16

# I/O and serialization
spark.serializer                      org.apache.spark.serializer.KryoSerializer
spark.kryoserializer.buffer.max       256m
spark.sql.files.maxPartitionBytes     134217728
spark.sql.files.openCostInBytes       4194304

# Fault tolerance
```

```
spark.task.maxFailures                4
spark.speculation                     true
spark.speculation.multiplier          1.5

# S3 (direct read/write)
spark.hadoop.fs.s3a.impl              org.apache.hadoop.fs.s3a.S3AFileSystem
spark.hadoop.fs.s3a.fast.upload       true
spark.hadoop.fs.s3a.multipart.size    67108864
spark.hadoop.fs.s3a.connection.maximum 100
```

7.3 Parameter tuning for horizontal scalability experiments

Parameter	2 core nodes	3 core nodes	4 core nodes	5 core nodes
spark.executor.instances	2	3	4	5
spark.default.parallelism	16	24	32	40
spark.sql.shuffle.partitions	16	24	32	40
Total available cores	4	6	8	10
Total executor RAM	12 GiB	18 GiB	24 GiB	30 GiB

8 Project S3 Bucket Structure

All project artifacts are stored in a dedicated S3 bucket (referred to as s3://proyecto-anomalias-emr/ for the purposes of this specification). The directory structure ensures separation between input data, trained models, streaming checkpoints, and evaluation results.

```
s3://project-anomalies-emr/
├── data/
│   ├── cic-ids2017/
│   │   ├── train/
│   │   │   ├── Monday-WorkingHours.pcap_ISCX.csv
│   │   │   ├── Wednesday-WorkingHours.pcap_ISCX.csv
│   │   │   └── Friday-WorkingHours.pcap_ISCX.csv
│   │   └── test/
│   │       ├── Tuesday-WorkingHours.pcap_ISCX.csv
│   │       └── Thursday-WorkingHours.pcap_ISCX.csv
│   └── csc-cic-ids2018/
│       └── (reference:  s3://cse-cic-ids2018/Processed Traffic Data/)
├── models/
│   ├── random_forest_pipeline/
│   └── kmeans_pipeline/
├── checkpoints/
│   ├── rf_streaming/
│   └── kmeans_streaming/
├── results/
│   ├── batch_evaluation/
│   ├── streaming_metrics/
│   └── scalability/
├── logs/
│   └── spark_event_logs/
└── scripts/
    ├── data_management_cic_ids2017.py
    ├── data_management_cic_ids2018.py
    ├── pipeline_random_forest.py
    └── pipeline_kmeans.py
```

```
├── tcp_sender.py
├── streaming_demo.py
└── monitor.py
```

9 Spark Structured Streaming Configuration

The streaming parameters define the behavior of the near real-time pipeline during the demonstration phase and are determinant for the reported latency and throughput metrics.

Parameter	Configured value	Description
Trigger interval	processingTime = '10s'	Duration of each micro-batch. Defines the temporal resolution of detection
Checkpoint location	s3://proyecto-anomalias-emr/checkpoints/	Storage of query offsets and state for fault tolerance.
Output mode	append	Only the new rows from each micro-batch are written.
maxFilesPerTrigger	5	Maximum number of CSV files consumed per micro-batch from S3.
Watermark (event time)	30 seconds	Tolerance for late data in the stream.
sink	console + S3 parquet	Real-time visualization and persistence of predictions.
spark.sql.streaming.metricsEnabled	true	Exposes throughput and latency to Spark UI and EMR Ganglia.

9.1 End-to-end latency measurement

Detection latency is defined as the time elapsed between the arrival timestamp of the first record of a micro-batch at the Spark pipeline (t_{ingest}) and the moment the prediction becomes available at the S3 sink ($t_{predict}$). This measurement is performed using the batchId, numInputRows, triggerExecution, and inputRowsPerSecond fields exposed by StreamingQueryListener, implemented as an observer in the script 05_streaming_demo.py.

10 Cluster Provisioning

10.1 AWS CLI

The following AWS CLI command exactly reproduces the base cluster configuration (1 master + 2 core nodes). It must be executed from a session with IAM credentials that have EMR, EC2, and S3 permissions.

```
aws emr create-cluster \
  --name "ProjectAnomalies" \
  --release-label emr-7.1.0 \
  --region us-east-1 \
  --applications \
      Name=Spark \
      Name=Hadoop \
      Name=JupyterEnterpriseGateway \
  --instance-groups \
      InstanceGroupType=MASTER,InstanceType=m5.xlarge,InstanceCount=1 \
      InstanceGroupType=CORE,InstanceType=m5.xlarge,InstanceCount=2 \
```

```
  --ec2-attributes \
      KeyName=<EC2_KEY_PAIR>,SubnetId=<SUBNET_ID> \
  --service-role EMR_DefaultRole \
  --ec2-attributes \
      InstanceProfile=EMR_EC2_DefaultRole \
  --log-uri s3://project-anomalies-emr/logs/emr/ \
  --bootstrap-actions \
      Path=s3://project-anomalies-emr/scripts/bootstrap.sh \
  --configurations file://emr-spark-config.json \
  --enable-debugging \
  --auto-terminate
```

10.2 emr-spark-config.json file

```
[
  {
    "Classification": "spark-defaults",
    "Properties": {
      "spark.executor.cores":                  "2",
      "spark.executor.memory":                 "6g",
      "spark.executor.memoryOverhead":         "1024",
      "spark.driver.memory":                   "6g",
      "spark.driver.cores":                    "2",
      "spark.executor.instances":              "2",
      "spark.default.parallelism":             "16",
      "spark.sql.shuffle.partitions":          "16",
      "spark.serializer":                      "org.apache.spark.serializer.KryoSerializer
    ",
      "spark.kryoserializer.buffer.max":       "256m",
      "spark.memory.fraction":                 "0.75",
      "spark.memory.storageFraction":          "0.40",
      "spark.task.maxFailures":                "4",
      "spark.sql.streaming.metricsEnabled":    "true",
      "spark.hadoop.fs.s3a.fast.upload":       "true"
    }
  },
  {
    "Classification": "spark-env",
    "Configurations": [{
      "Classification": "export",
      "Properties": { "PYSPARK_PYTHON": "/usr/bin/python3" }
    }]
  }
]
```

11 AWS Region and Network Configuration

us-east-1 (N. Virginia) Co-location with the public CSE-CIC-IDS2018 bucket

Network parameter	Value	Justification
VPC / subnet	Default VPC (public)	Simplified access for the academic context.
Security groups	ElasticMapReduce-master / -slave	Default EMR groups; allow internal communication between nodes and SSH access from an IP.
EC2 key pair	Generated specifically for the project	Allows SSH access to the master node for manual inspection and debugging.
S3 $\Longleftrightarrow$ EMR transfer	No additional cost	Within the same region; S3 VPC Gateway Endpoint configured by default in EMR.

References

[1] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal.

[2] Communications Security Establishment (CSE) & Canadian Institute for Cybersecurity (CIC). (2018). *CSE-CIC-IDS2018 on AWS*. Registry of Open Data on AWS. https://registry.opendata.aws/cse-cic-ids2018

[3] Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., ... & Xin, R. (2016). *MLlib: Machine learning in Apache Spark*. Journal of Machine Learning Research, 17(1), 1235–1241.

[4] Armbrust, M., Das, T., Torres, J., Yavuz, B., Zhu, S., Xin, R., ... & Zaharia, M. (2018). *Structured streaming: A declarative API for real-time applications in Apache Spark*. Proceedings of the ACM SIGMOD International Conference on Management of Data.

[5] AWS. (2026). *Apache Spark on Amazon EMR*. Apache Spark on EMR documentation. https://aws.amazon.com/emr/features/spark/